Agentic Security System

Full technical deep dive: architecture, LangGraph workflow, ML integration, LLM planning, deception pipeline, runtime behavior, demos, and known limitations.

LangGraph Orchestration | Host Monitoring | ML Risk Scoring | LLM Analysis | Deception Strategy | File-Focused Response

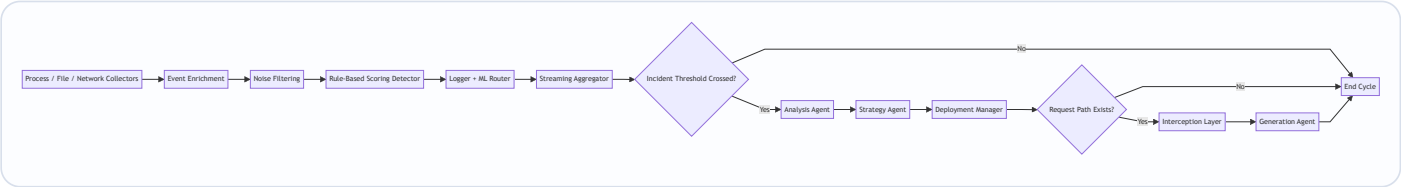
> **Core idea:** the system monitors host telemetry, detects suspicious behavior, estimates risk with ML, reasons about attacker intent with LLMs, plans deception, deploys decoys, and conditionally serves fake content through its interception flow.

1. Executive Summary

The project is a layered security pipeline built around one explicit orchestration graph. It monitors process, file, and network activity, derives enriched behavioral features, filters noise, scores suspicious events, routes events into specialized ML models, aggregates risk, and only then escalates into LLM-based analysis and deception planning.

The strongest and most complete part of the system is the **file-side deception path**: strategy planning, decoy deployment metadata, file interception decisions, and realistic fake content generation. LangGraph is used as the control plane that ties these modules together into a single, testable workflow.

2. High-Level Architecture



The left half is detection and risk aggregation. The right half is deception planning and controlled response.

3. Active Runtime Path

The dominant executable path in the repository is:

`main.py -> SystemAgent -> LangGraphSecurityPipeline -> monitor -> enrich -> filter -> score -> emit_alerts -> analysis -> strategy -> deployment -> interception`

Primary runtime files

- `main.py` : startup and config loading
- `agents/system_agent.py` : long-running live loop
- `langgraph_pipeline.py` : orchestration graph
- `state_schema.py` : unified shared state

Main subsystems

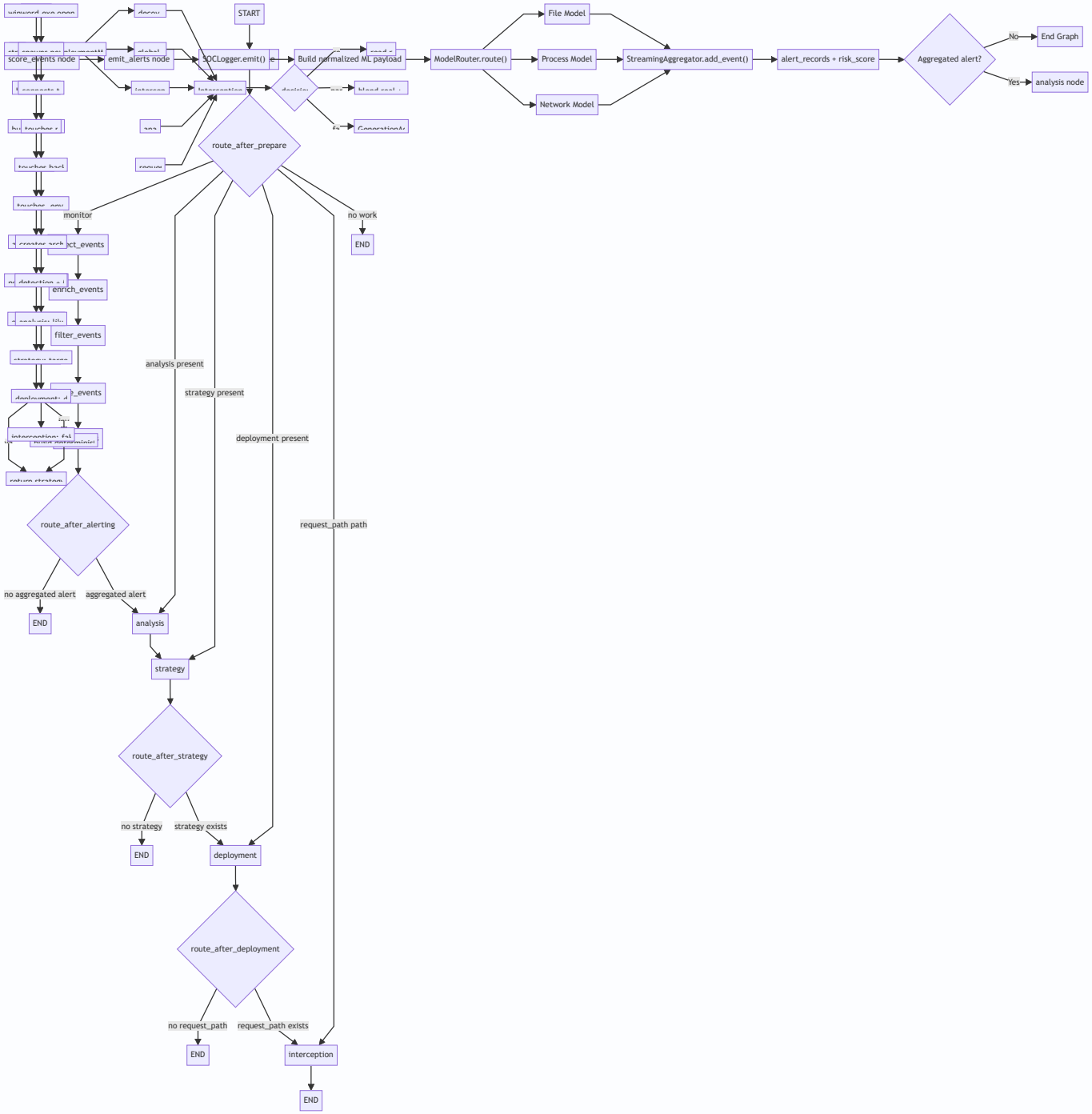
- `collectors/` : telemetry ingestion
- `detectors/` and `logs/` : detection and ML handoff
- `agents/analysis` and `agents/strategy` : LLM reasoning
- `agents/deployment` , `core/interception_layer.py` , and `agents/generation` : deception response

4. LangGraph State

The graph is built over a shared state object defined in `state_schema.py`. Every node reads what it needs and writes its output back to this one structure.

State Field	Purpose
`mode`	Controls whether the graph runs a full monitoring cycle or an interception-oriented path.
`input_events`	Optional externally supplied events for demos, tests, or forced executions.
`raw_events`	Collected telemetry straight from the monitor.
`enriched_events`	Events after feature engineering.
`filtered_events`	Events after noise suppression.
`detections`	Detection records containing event, rule output, and acceptance decision.
`suspicious_events`	Accepted suspicious events.
`alert_records`	Logger + ML router + aggregator outputs.
`risk_score`	Dominant aggregated incident score.
`analysis`	Attacker intent/stage/confidence interpretation.
`strategy`	Executable deception plan.
`strategy_meta`	Strategy provenance and fallback metadata.
`deployment`	Decoy registry, context, and interception rules.
`request_path`	Requested file path for interception mode.
`interception_result`	Final real, partial, or fake content returned by the interception flow.
`errors`, `notes`, `cycle_report`	Operational traces and live reporting.

5. LangGraph Nodes and Edges



Node responsibilities

Node	Reads	Writes	Wrapped component
`prepare_state`	incoming state	reset cycle fields	internal state initializer
`collect_events`	`input_events` or monitor	`raw_events`	`core.monitor.Monitor`

Node	Reads	Writes	Wrapped component
`enrich_events`	`raw_events`	`enriched_events`	`agents.event_enrichment.EventEnricher`
`filter_events`	`enriched_events`	`filtered_events`	`utils.filters.EventFilter`
`score_events`	`filtered_events`	`detections`, `suspicious_events`	`detectors.scoring.ScoringDetector`
`emit_alerts`	`detections`	`alert_records`, `risk_score`	`logs.logger.SOCLogger`
`analysis`	`alert_records`, `risk_score`	`analysis`	`agents.analysis.analysis_agent`
`strategy`	`analysis`	`strategy`, `strategy_meta`	`agents.strategy.strategy_agent`
`deployment`	`strategy`	`deployment`	`agents.deployment.deployment_agent.DeploymentManager`
`interception`	`request_path`, `analysis`, `deployment`	`interception_result`	`core.interception_layer.InterceptionLayer`

6. Detection and ML Integration

LangGraph does not directly run the models itself. Instead, the graph reaches the ML layer through the `emit_alerts` node, which calls the logger. The logger builds normalized ML payloads, routes them to the correct model, and sends outputs into the streaming aggregator.

Why this design matters

- Rule-based detection catches suspicious patterns quickly.
- Specialized models score different event categories differently.
- The aggregator prevents single weak anomalies from always escalating into expensive LLM reasoning.

7. Analysis Agent

The analysis agent takes aggregated suspicious events and infers attacker intent, attack stage, confidence, and reasoning. It acts as the semantic bridge between raw risk and strategic response.

Inputs

- `risk_score`
- aggregated suspicious events

Outputs

- `intent`
- `attack_stage`
- `confidence`
- `reasoning`

Typical intent examples include `data_exfiltration`, `lateral_movement`, and `reconnaissance`. Typical stage values include `collection`, `exfiltration`, and `credential_access`.

8. Strategy Agent

The strategy agent is the planning layer. It takes analysis output and converts it into a structured deception blueprint. It does not create files directly; it decides what should exist, where it should be placed, how much attacker interaction should be allowed, and what should be monitored.

Strategy module files

File	Role
`agents/strategy/strategy_agent.py`	Main controller for the planning pipeline.
`agents/strategy/prompt_builder.py`	Builds deterministic hints and the strict JSON prompt.
`agents/strategy/parser.py`	Extracts JSON from model output.
`agents/strategy/schema.py`	Defines allowed planning values and generation limits.
`agents/strategy/validator.py`	Repairs, validates, constrains, and falls back safely.

> **Key design decision:** the LLM is allowed to propose a plan, but deterministic policy decides the final safe output.

9. Deployment and Interception

Deployment converts the strategy blueprint into operational metadata. Interception consumes that metadata plus attacker analysis to decide whether a file request should return real content, blended content, or fake content.

The most mature response surface in the project is the file side. The deployment registry and interception logic are designed mainly around decoy file artifacts.

10. SystemAgent Runtime

`SystemAgent` is the long-running runtime controller. It is responsible for continuously invoking the LangGraph workflow and printing live incident summaries.

Responsibilities

- load monitoring configuration
- configure watched file paths
- construct `LangGraphSecurityPipeline`
- run repeated monitoring cycles
- print human-readable cycle reports

Live cycle reporting

Each interesting cycle can report:

- how many events were seen
- event type distribution
- file paths observed
- suspicious file targets
- aggregated risk score
- analysis summary
- deployed decoys

11. File-Focused Realistic Attack Story

The most presentation-friendly scenario is a file-heavy intrusion sequence, because it exercises the strongest parts of the system.

This exact style of scenario is demonstrated in the file-focused walkthrough script and is the best way to show the complete system end to end.

12. End-to-End Validation Workflow

The repository contains a one-command PowerShell validation script that checks the major paths of the system.

Validation covers

- environment prerequisites
- model artifact presence
- LLM config consistency
- strategy tests
- deployment tests
- pipeline demo
- attacker simulation
- watcher demo
- LangGraph full cycle
- interception mode
- main startup smoke test

Main commands

- `.\run_all_checks.ps1`
- `python presentation_demo.py`
- `python presentation_demo_file_attack.py`
- `python tests\run_event_pipeline.py`

13. Strengths

Explicit orchestration

LangGraph makes the workflow visible, conditional, and state-driven instead of implicit.

Layered reasoning

Rules, ML, and LLMs each serve distinct roles rather than competing with each other.

Strong file response path

The project demonstrates coherent file-focused deception from planning to fake content generation.

14. Limitations and Loopholes

Important: this system is not a kernel-level, tamper-proof, full-OS interception product.

- If an attacker gains complete administrative or kernel-level control of the OS, a user-space orchestration system cannot guarantee persistence.
- The strongest response path is file-based deception. Other response surfaces are less complete.
- The interception flow is controlled by application logic, not by a Windows minifilter or universal filesystem hook.
- The repository still contains an older `src/` scaffold, which is not the real runtime path.
- LLM quality and availability affect the quality of analysis and strategy generation, although safe fallbacks reduce that risk.

Best way to answer this in defense

If asked how the system persists after full OS compromise, the correct answer is:

> "If the attacker has full OS control, a user-space system cannot be assumed fully trustworthy. The value of this project is early detection, deception, and visibility before that point, not guaranteed survival under total host compromise."

15. Future Improvements

- physically materialize deployed decoys into configured real watch folders by default
- introduce protected remote logging and remote orchestration
- expand deception beyond files into stronger network and endpoint surfaces
- add kernel-level or gateway-level interception if full transparent file substitution is required
- improve packaging and documentation around the active runtime versus legacy scaffolding

16. Final Summary

This system is best described as a **LangGraph-orchestrated host monitoring and deception platform**. Its main contribution is not a single detector or single agent, but the way it unifies telemetry collection, risk estimation, semantic reasoning, deception planning, deployment, and controlled response into one coherent workflow.

In practical terms, it detects suspicious behavior, estimates attacker risk, reasons about likely intent, plans targeted decoys, and can serve believable fake artifacts in a file-focused defense scenario. That is the full system story.